

代码片段 6.19 clone 的伪代码实现

---

```

1 int clone(..., int flags, ...)
2 {
3     // 创建一个新的 PCB, 用于管理新进程
4     struct process *new_proc = alloc_process();
5
6     // 如果设置了 CLONE_VM, 则直接使用父进程的页表, 否则拷贝一份
7     if (flags & CLONE_VM) {
8         new_proc->vmSPACE->pgtbl =
9             curr_proc->vmSPACE->pgtbl;
10    } else {
11        new_proc->vmSPACE->pgtbl = alloc_page();
12        copy_vmSPACE(new_proc->vmSPACE,
13                     curr_proc->vmSPACE);
14    }
15
16    // 内核栈初始化
17    init_kern_stack(new_proc->stack);
18    // 上下文初始化: 将父进程 PCB 中的上下文完整拷贝一份
19    copy_context(new_proc->ctx, curr_proc->ctx);
20
21    // 如果设置了 CLONE_VFORK, 则使父进程在内核中等待
22    if (flags & CLONE_VFORK) {
23        while (!exec_or_exit(new_proc))
24            schedule();
25    }
26    // 返回
27 }

```

---

### 小知识: Windows 的进程创建——CreateProcess

Windows 采用的进程创建接口为 CreateProcess 系列。其中语义比较简单的是 CreateProcessA, 该函数原型如下:

---

```

1 #include <Processthreadsapi.h>
2
3 BOOL CreateProcessA(
4     LPCSTR          lpApplicationName,
5     LPSTR           lpCommandLine,
6     LPSECURITY_ATTRIBUTES lpProcessAttributes,
7     ...             // 其他七个配置参数
8 );

```

---

CreateProcess 与 posix\_spawn 以及前面介绍的 process\_create 存在一定的相似之处, 也采取了从头创建进程的方式, 载入参数 lpApplicationName 指定的可执行文件, 并根据其他参数的配置返回用户态执行。但为了支持在创建进程

时对其进行多种多样的配置，CreateProcess 的接口要比 posix\_spawn 更加复杂，就算是较为简单的 CreateProcessA 接口也需要传入 10 个参数。

## 练习

6.4 posix\_spawn 和 fork 都具有创建进程的功能，但语义有所不同。请分析在以下三种情况下 fork 和 posix\_spawn 哪个更适合，并解释原因。

- Web 服务器接收到请求，并创建一个新进程处理该请求。
- shell 接收到用户输入的 ls 命令，并创建一个新进程来执行该命令。
- 父进程创建一个子进程，并希望设置共享内存来进行进程之间的通信。

## 6.4 进程切换

### 本节主要知识点

- 进程的处理器上下文中包含哪些内容？
- 进程切换包含哪些步骤？
- ChCore 中的进程切换是如何实现的？

现代操作系统通过分时复用的方法，允许不同进程轮番使用处理器，营造出了“多个进程同时执行”的假象。为此，操作系统提出了进程切换机制，从一个进程（原进程）的“上下文”切换到另一个进程（目标进程）的“上下文”执行。在切换过程中，由于进程的重要状态保存在处理器的寄存器中，因此在切换进程前需要保存这些状态（即处理器上下文），以便之后恢复执行。又因为处理器上下文等进程相关的信息都保存在内核中，所以进程切换需要在内核中完成。总的来说，进程切换的主要工作流包括：原进程进入内核态→保存原进程的处理器上下文→切换进程上下文（切换虚拟地址空间和内核栈）→恢复目标进程的处理器上下文→目标进程返回用户态（如图 6.6 所示）。

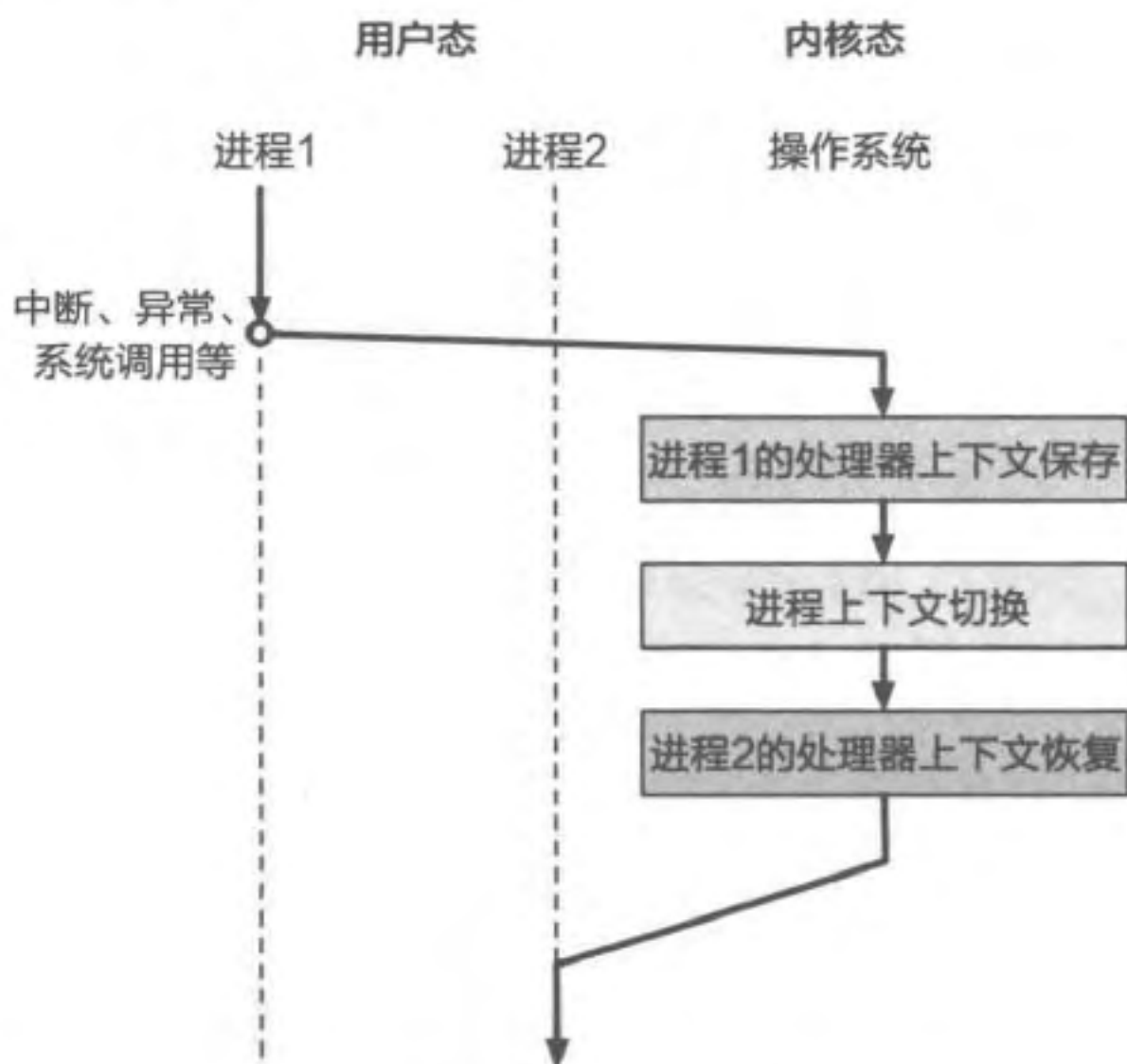


图 6.6 进程切换包含的主要工作流



## 进程上下文与处理器上下文

本节出现了两个上下文的概念：进程上下文和处理器上下文。它们存在什么差别呢？

进程上下文是操作系统提供的软件概念，表示操作系统目前正以哪个进程的身份运行。也就是说，操作系统在为哪个进程填充页表、处理中断，消耗哪个进程的时间片，那么它就处于哪个进程的上下文中。进程上下文在内核中常以一个指向当前运行进程 PCB 的指针形式出现（比如本章中一直使用的 `curr_proc` 变量），而 PCB 中包含的所有内容都属于进程上下文的范畴。而相对地，处理器上下文则是一个硬件概念，它包含的是处理器中当前寄存器的状态。由于进程运行过程中的状态也包含处理器中的寄存器状态，因此处理器上下文是进程上下文的一部分。

### 6.4.1 进程的处理器上下文

代码片段 6.20 展示了进程处理器上下文数据结构（`context`）包含的内容。根据第 3 章的分析，`context` 包含以下寄存器中的值：

- 所有通用寄存器（`X0 ~ X30`）。
- 特殊寄存器中的用户栈寄存器 `SP_EL0`。该寄存器并不会被硬件自动保存，所以需要手动存储，以便下次切换时恢复到正确的栈顶地址。
- 系统寄存器中的 `ELR_EL1` 和 `SPSR_EL1`。这两个寄存器在从用户态切换到内核态时分别保存了 PC 寄存器和 `PSTATE` 的值，因此保存它们可以使处理器硬件状态和程序计数器在进程切换后得到正确恢复。

代码片段 6.20 进程处理器上下文数据结构 `context` 包含的内容

```
1 // 进程处理器上下文内部包含的内容
2 struct context {
3     // 通用寄存器
4     u64 x0, x1, ..., x30;
5     // 特殊寄存器
6     u64 sp_el0;
7     // 系统寄存器
8     u64 elr_el1, spsr_el1;
9 };
```

### 6.4.2 进程的切换节点

一般来说，进程切换的触发方式可以分为主动和被动两种。主动切换指进程主动放弃 CPU 资源，并交给其他进程使用。当进程主动调用 `process_exit`、`process_waitpid`、`process_sleep` 等系统调用时，最终会调用 `schedule` 函数，使操作系

统调度下一个进程执行，这种情况属于主动切换。被动切换则是由操作系统强制触发的，通常基于硬件中断实现：当中断触发时，控制流将转移到内核，因此可以借机进行切换。一种常见的被动切换触发方式是利用时钟中断实现的，我们将在之后的案例分析中详细介绍。

6.4.3 进程切换的全过程

图 6.7 总结了从进程 p0 切换到进程 p1 的过程中，处理器与内存状态的变化过程。具体来说，进程切换可以分为以下六个步骤。

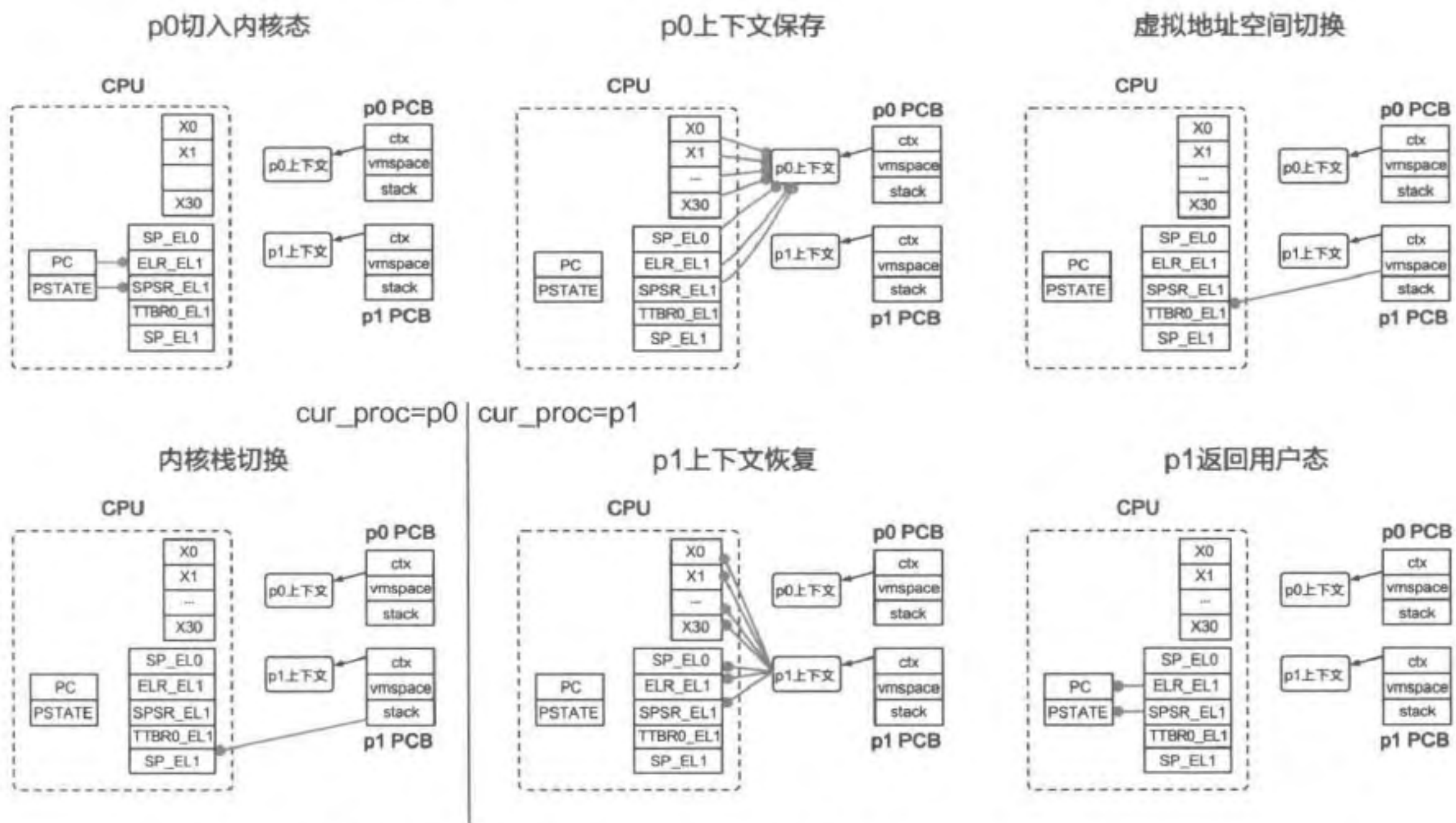


图 6.7 进程切换的全过程。圆箭头表示每一步的具体赋值

1. p0 从用户态进入内核态（通过系统调用、异常、中断等方法），此时硬件会自动将 PC 和 PSTATE 寄存器的值分别保存到 ELR\_EL1 和 SPSR\_EL1 寄存器中。
2. 内核获取 p0 的处理器上下文结构，并将前述寄存器依次保存到处理器上下文中。
3. 内核获取 p1 页表基地址（保存在 PCB 内部的 vm space 结构中），并存储到 TTBR0\_EL1 寄存器中，即完成了虚拟地址空间的切换。由于这一步涉及虚拟地址空间的变动，可能需要添加 TLB 刷新的指令，防止后续执行时的地址翻译错误。
4. 内核将 SP\_EL1 切换到进程 p1 私有的内核栈顶地址，从而完成内核栈的切换。至此，内核将不再访问任何与 p0 相关的数据，因此可以认为内核栈切换是进程切换的关键分界点，内核此时可以将 `curr_proc` 设置为 p1，从而完成进程上下文的切换。
5. 内核从 p1 的 PCB 中获取其处理器上下文结构，并依次恢复到前述寄存器中。



6. 内核执行 `eret` 指令返回用户态，此时硬件会自动将 `ELR_EL1` 和 `SPSR_EL1` 寄存器中的值恢复到 `PC` 和 `PSTATE` 中。至此，进程切换完成，`p1` 将恢复执行。

## 练习

6.5 结合进程切换场景回答：为什么每个进程都要有自己的内核栈？

### 6.4.4 案例分析：ChCore 的进程切换实现

前面的章节已经介绍了进程切换包含的具体步骤，但其实现并不是那么直观，有很多细节需要留意。为了展示进程切换的具体实现，本节将以 ChCore 为例介绍一种简化的实现方案。

#### 切换过程总览

前面已经介绍过，进程切换的触发方式可分为主动和被动两种。主动触发通常是通过系统调用进入内核态，第 3 章已经对该过程进行了详细介绍。因此，本节将主要以时钟中断的处理为例详细介绍 ChCore 中进程被动切换的步骤，并简述主动切换与之存在的差别。

图 6.8 展示了时钟中断发生后 ChCore 中进程切换的全过程。当中断来临后，无论进程在用户态执行什么代码，处理器都会进行与系统调用类似的保存处理器状态的工作（详见 3.2 节），下陷到内核态，并跳转到异常向量表中对应中断的条目（在本例中为 `irq_el0_64`）。该条目首先调用 `exception_enter` 执行处理器上下文的保存，随后进入 `handle_irq` 函数执行时钟中断的具体逻辑，然后调用在前面章节多次提到的 `schedule` 函数。注意，由于 ChCore 中的调度策略比 6.1.8 节介绍的更为复杂，因此 `schedule` 函数的实现也有所不同。在 ChCore 的实现中，`schedule` 函数首先调用 `sched` 选出下一个需要执行的进程（即目标进程），然后通过 `eret_to_process` 切换到目标进程，并返回用户态执行。接下来将对这些步骤进行详细分析。

#### 处理器上下文保存

为了便于实现处理器上下文保存及之后的恢复功能，ChCore 对内核中的数据结构进行了调整。由于每个进程有专门的内核栈，ChCore 直接将进程的处理器上下文放入栈中。图 6.9 展示了 ChCore 中 PCB (`cap_group`)、处理器上下文及内核栈三种数据结构的关系<sup>①</sup>，不难看出，进程的处理器上下文位于其内核栈底部。正是由于处理器上下文处于内核栈底，对其的保存只需将需要保存的寄存器值依次放入进程对应的内核栈中即可。代码片段 6.21 展示了 `exception_enter` 的实现方法。

<sup>①</sup> `cap_group` 的虚拟地址不一定高于内核栈和处理器上下文，图 6.9 仅为示例。

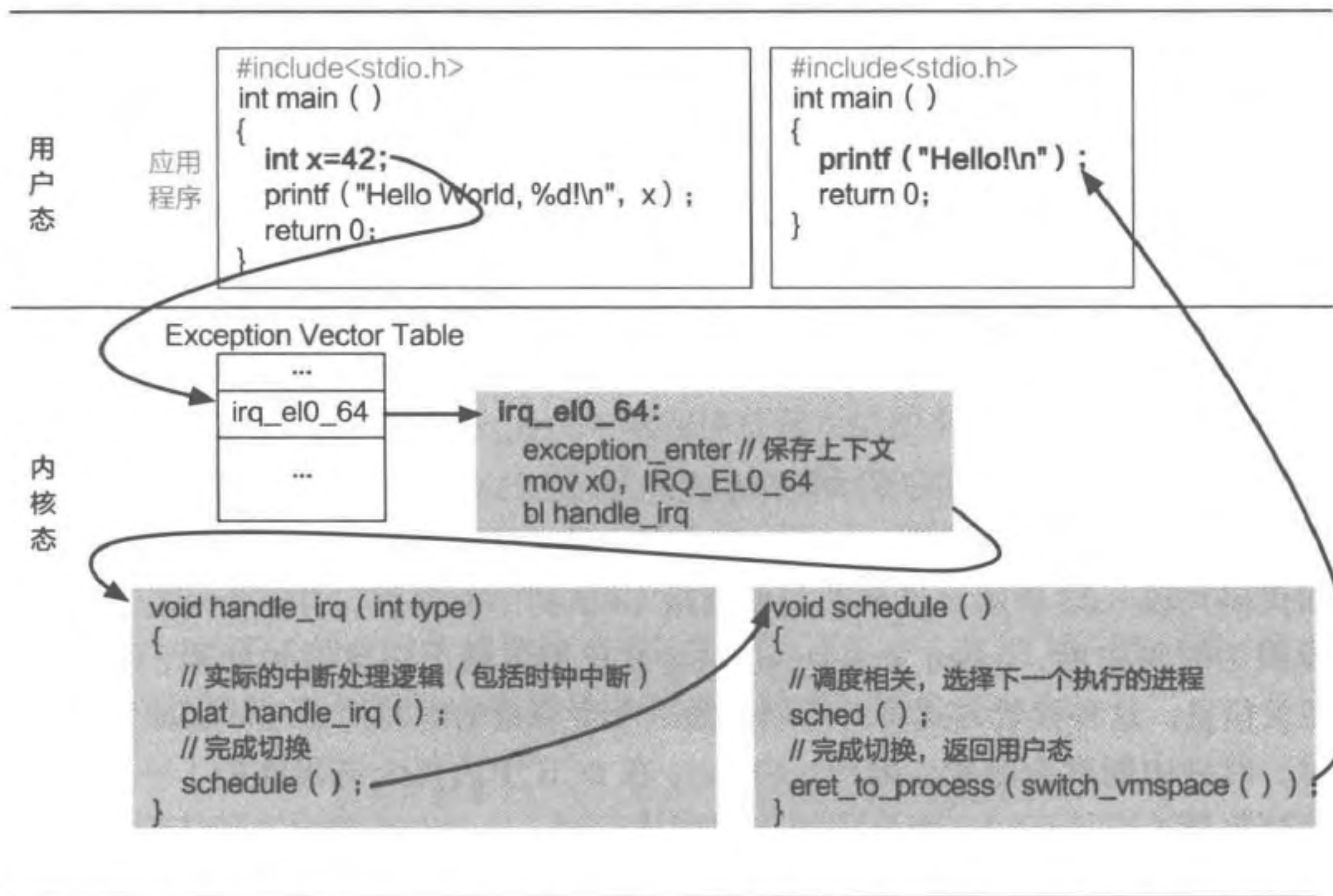


图 6.8 AArch64 体系结构下 ChCore 由时钟中断触发进程切换的例子

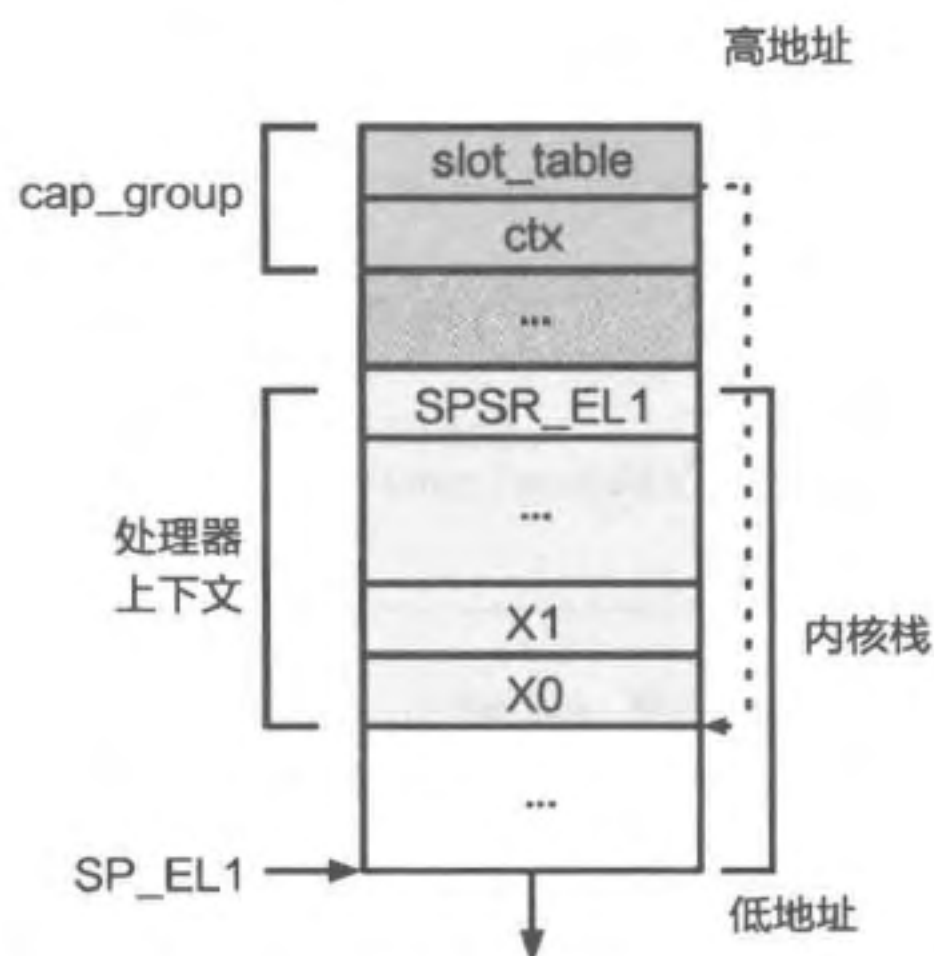


图 6.9 ChCore 中进程切换相关的数据结构

代码片段 6.21 保存进程的处理器的上下文时 `exception_enter` 使用的代码

```
1 .macro exception_enter
2   sub sp, sp, #ARCH_EXEC_CONT_SIZE
3   // 保存通用寄存器 (x0-x29)
4   stp x0, x1, [sp, #16 * 0]
5   stp x2, x3, [sp, #16 * 1]
6   stp x4, x5, [sp, #16 * 2]
```



```

7   ...
8   stp x28, x29, [sp, #16 * 14]
9   // 保存 x30 和上文提到的三个特殊寄存器: sp_el0, elr_el1, spsr_el1
10  mrs x21, sp_el0
11  mrs x22, elr_el1
12  mrs x23, spsr_el1
13  stp x30, x21, [sp, #16 * 15]
14  stp x22, x23, [sp, #16 * 16]
15 .endm

```

### 中断处理与进程调度

在执行 `exception_enter` 后，操作系统进入 `handle_irq` 函数，其实现如图 6.8 所示。

首先，`handle_irq` 调用 `plat_handle_irq` 函数，负责与中断相关的处理逻辑。如代码片段 6.22 所示，该函数获取当前 CPU 的中断原因，并进行相应处理。如果中断原因为时钟中断，`plat_handle_irq` 会首先更新下一次时钟间隔，并维护进程调度相关信息。这种设置方式将时间划分为一个个等量的时间片（time slice），当时间片用尽时，时钟中断就会触发。因此，ChCore 在 PCB 中为每个进程维护了一个当前剩余的时间片数量（budget）。当进程刚被调度执行时，budget 被设为默认值 `DEFAULT_BUDGET`。而在代码片段 6.22 中，由于当前时间片已经耗尽，因此当前进程剩余的时间片会减一，若减到 0，就会暂停执行，并让调度器选择下一个需要执行的进程。

代码片段 6.22 ChCore 的时钟中断处理逻辑

```

1 void plat_handle_irq(void)
2 {
3     u32 cpuid = 0;
4     unsigned int irq_src, irq;
5     // 获取当前 CPU 及其中断原因
6     cpuid = smp_get_cpu_id();
7     irq_src = get32(core_irq_source[cpuid]);
8     irq = 1 << ctzl(irq_src);
9
10    // 根据不同原因进行处理
11    switch (irq) {
12        case INT_SRC_TIMER3:
13            // 更新下一次时钟中断间隔
14            asm volatile ("msr cntv_tval_el0, %0"::"r"(cntv_tval));
15            // 更新剩余的时间片数量
16            if (curr_proc->budget > 0)
17                curr_proc->budget--;
18            break;
19
20            // 处理其他中断
21            case ...:
22        }
23    return;
24 }

```

之后，`handle_irq` 调用 `schedule` 函数并进入 `sched` 函数中，执行调度相关逻辑。`sched` 利用操作系统的调度器选取下一个需要执行的进程，作为切换的目标进程。一种基于时间片数量的简单 `sched` 实现方法如代码片段 6.23 所示。首先，`ChCore` 需要判断 `curr_proc` 是否为空，这是因为在主动切换的情况下，前一个执行的进程可能已经调用了 `process_exit`，不可能再次被调度，所以在 `process_exit` 中会将 `curr_proc` 设为空（代码片段 6.3 中的伪代码省略了这部分细节）。然后，`ChCore` 会读取其剩余时间片数量 `budget`，若时间片还有剩余则不需要切换，直接返回，否则执行调度策略，选择下一个进程，并将 `curr_proc` 指向该进程对应的 PCB。该选择策略多种多样，最简单的策略可以像代码片段 6.13 中 `pick_next` 描述的那样扫描进程列表，找到第一个执行状态为 `READY`（就绪）的进程（第 7 章还将介绍更多常见的调度策略）。最后，`ChCore` 为下个要执行的进程配置时间片，然后返回。需要注意的是，虽然这里 `curr_proc` 已经被切换了，但此时内核还处于原进程的上下文中，直到下一个步骤（虚拟地址空间和内核栈切换），这种设计主要还是出于简化实现的考虑。

代码片段 6.23 ChCore 中调度相关逻辑的实现

---

```

1 int sched(void)
2 {
3     // 如果 curr_proc 不为空，且时间片还未用尽，则直接返回
4     if (curr_proc && curr_proc->budget != 0)
5         return 0;
6
7     // 已经用尽，选择下个进程执行
8     curr_proc = pick_next();
9     // 将选中的进程执行状态变为运行
10    curr_proc->exec_status = RUNNING;
11    // 为下个进程配置时间片数量，然后返回
12    curr_proc->budget = DEFAULT_BUDGET;
13    return 0;
14 }

```

---

除了时钟中断引发的进程切换之外，`ChCore` 中还存在因系统调用引发的主动切换。前面已经提到，由于调用 `process_exit` 的进程已经退出，不应该再次被调度执行，因此需要将 `curr_proc` 置为空。除了 `process_exit` 之外，还有一类使进程陷入等待的系统调用（如 `process_waitpid`）。对于陷入等待的进程，在其等待的条件（如子进程退出）满足之前，不应该调度它们执行，否则只会浪费 CPU 资源。因此，`ChCore` 要求这些系统调用在进入 `schedule` 函数前，将当前进程拥有的时间片置为 0，表明放弃之后的时间片，请求内核调度下一个进程执行。代码片段 6.24 以 `process_sleep` 为例展示了兼容 `ChCore` 调度机制的实现方式，`process_waitpid` 也可以采取类似方法实现。



代码片段 6.24 进程睡眠的伪代码实现 (第二版): 在调度下个进程前将时间片置为 0

---

```

1 void process_sleep(int seconds)
2 {
3     // 获取当前时间作为睡眠起始时间
4     struct *date start_time = get_time();
5     while (TRUE) {
6         struct *date cur_time = get_time();
7         if (time_diff(cur_time, start_time) < seconds) {
8             // 如果时间未到, 则将当前进程的时间片置为 0
9             curr_proc->budget = 0;
10            // 调度下个进程执行
11            schedule();
12        } else {
13            // 时间已到, 直接返回
14            return;
15        }
16    }
17 }

```

---

### 虚拟地址空间与内核栈切换

在完成调度相关逻辑之后, 内核将进入 `switch_vmspace` 函数, 其主要包含两个步骤, 如代码片段 6.25 所示。首先, 为了实现虚拟地址空间的切换, 该函数从目标进程的 PCB 中获取页表基地址, 并将其转为物理地址, 最后设置到 `TTBR0_EL1` 寄存器中。接着, 该函数返回目标进程的处理器上下文所在地址, 之后作为参数传给 `eret_to_process`。

代码片段 6.25 `switch_vmspace` 函数的实现

---

```

1 void* switch_vmspace(void)
2 {
3     // 切换虚拟地址空间: 获取页表所在物理地址并设置
4     set_ttbr0_el1(
5         virt_to_phys(curr_proc->vmspace->pgtbl));
6     // 返回处理器上下文所在地址
7     return curr_proc->ctx;
8 }

```

---

之后, `eret_to_process` 函数将完成内核栈的切换。如代码片段 6.26 所示, 由于目标进程的处理器上下文所在地址已经作为参数保存在 `X0` 中, `eret_to_process` 函数将其移入栈寄存器 `SP_EL1` 中, 从而切换到目标进程的内核栈。这一步可能会让读者感到疑惑: 为什么会用目标进程的处理器上下文所在地址来实现内核栈切换呢? 但结合图 6.9 中内核栈与处理器上下文结构的关系就能发现, 由于处理器上下文本身就固定存储在内核栈底部, 因此只要切换到处理器上下文地址, 也就实现了内核栈切换。在这一步切换完成后, 栈寄存器 `SP_EL1` 的地址变为目标进程的处理器上下文所在地址, 即指向图 6.9 中寄存器 `X0` 的存储地址, 从而做好了处理器上下文恢复的准备。

代码片段 6.26 `eret_to_process` 函数的实现

---

```

1 BEGIN_FUNC(eret_to_process)
2 // 函数原型: void eret_to_process(u64 sp)
3 // 内核栈切换
4 mov sp, x0
5 // 进程切换的剩余步骤
6 exception_exit
7 END_FUNC(eret_to_process)

```

---

## 练习

6.6 如果调度器选择的下一个执行的进程和原进程恰好相同, 还需要执行 `switch_vmspace` 和 `eret_to_process` 吗?

### 处理器上下文恢复及返回用户态

完成内核栈切换后, `eret_to_process` 调用 `exception_exit` 完成进程切换的剩余步骤, 其实现如代码片段 6.27 所示。不难看出, `exception_exit` 的实现与 `exception_enter` 是完全对应的: 它将之前保存在内核栈中的值依次恢复到寄存器中, 并通过 `add` 指令将内核栈变为空。最后, `exception_exit` 调用 `eret` 指令返回用户态执行, 此时目标进程就会从处理器上下文中保存的指令处继续执行了。

代码片段 6.27 `exception_exit` 的实现

---

```

1 .macro exception_exit
2 // 恢复 x30 和三个特殊寄存器
3 ldp x22, x23, [sp, #16 * 16]
4 ldp x30, x21, [sp, #16 * 15]
5 msr sp_el0, x21
6 msr elr_el1, x22
7 msr spsr_el1, x23
8 // 恢复通用寄存器 X0-X29
9 ldp x0, x1, [sp, #16 * 0]
10 ...
11 ldp x28, x29, [sp, #16 * 14]
12 add sp, sp, #ARCH_EXEC_CONT_SIZE
13 eret
14 .endm

```

---

### 总结

图 6.10 仍以时钟中断为例, 完整展示了 ChCore 中从进程 `p0` 切换到进程 `p1` 过程中数据结构的变化。不难看出, 当内核栈切换完成后, ChCore 将只访问进程 `p1` 的状态, 因此可以认为内核栈切换是从 `p0` 进程上下文到 `p1` 进程上下文的切换节点。另外, 在返



回用户态时，进程私有的内核栈总是空的，这是因为操作系统已经完成了处理（无论是中断还是系统调用），无须保存任何栈帧。因此，当进程进入内核态时，就可以直接在内核栈底部保存处理器上下文，从而也保证了进程的处理器上下文地址始终固定在其内核栈底。

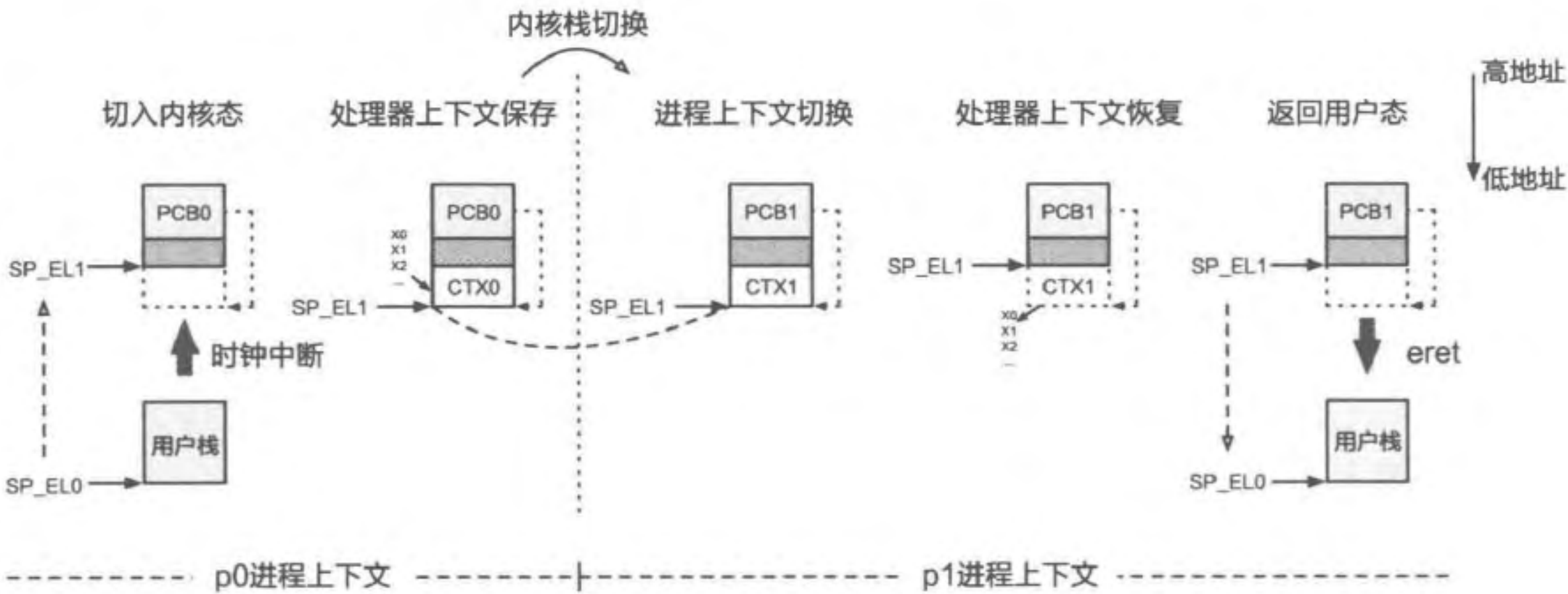


图 6.10 进程切换过程中数据结构的变化

## 练习

6.7 结合 ChCore 进程切换的过程回答：为什么 ChCore 的 PCB 中没有包含内核栈？

### 保留进程内核状态的实现

如果进程在内核执行的过程中也可能被切换，进程切换应该如何实现呢？这种情况最大的特点在于当进程下次恢复执行时，将会继续执行内核中的相关逻辑，因此它在内核中的状态（栈帧和寄存器）是有价值的，不能随意舍弃。以前面介绍的睡眠相关系统调用 `process_sleep` 为例（代码片段 6.11），当进程再次被内核调度，并从 `schedule` 函数中返回后，还需要继续在 `while` 循环中执行，不能直接返回用户态。因此，内核在切换前需要将该进程在内核中的处理器上下文保存起来，以便之后恢复执行。

由于需要保存和恢复内核中的处理器上下文，进程切换也变得更加复杂了。在进行内核栈切换前，进程需要首先保存自身在内核中的处理器上下文。而在内核栈切换后，新的进程也需要恢复内核中的处理器上下文，才能从上次切换的节点开始执行。为了维护这个“额外”的处理器上下文，PCB 需要引入 `kern_ctx` 字段。

代码片段 6.28 首先展示了修改后的 `schedule` 函数的实现方式，其逻辑与图 6.8 中

的版本相比，最主要的不同是调用了 `switch_to_process` 函数，而不是 `eret_to_process`。代码片段 6.29 展示了切换函数 `switch_to_process` 的实现，该函数负责内核栈以及内核中处理器上下文的切换。`switch_to_process` 需要接收两个参数，分别是原进程 PCB 的 `kern_ctx` 字段所在地址以及目标进程 PCB 的 `kern_ctx` 字段保存的地址。因此在调用 `sched` 之前，`schedule` 函数需要记录原进程的 PCB，以便之后将原进程的 `kern_ctx` 传入 `switch_to_process` 中完成切换。

代码片段 6.28 保留进程内核状态的 `schedule` 函数实现

---

```

1 void schedule_new()
2 {
3     // 保存原进程的 PCB 所在地址
4     struct process* prev_proc = curr_proc;
5     // 调度相关，选择下一个执行的进程
6     sched();
7     // 切换虚拟地址空间
8     switch_vmspace();
9     // 传入内核中的处理器上下文，完成进程切换
10    switch_to_process(&prev_proc->kern_ctx, curr_proc->kern_ctx);
11 }

```

---

代码片段 6.29 内核中切换函数 `switch_to_process` 的代码实现

---

```

1 BEGIN_FUNC(switch_to_process)
2     // 函数原型: void switch_to_process(struct context**, struct context*);
3     sub sp, sp, #ARCH_EXEC_CONT_SIZE
4     // 保存通用寄存器 X0-X30
5     stp x0, x1, [sp, #16 * 0]
6     stp x2, x3, [sp, #16 * 1]
7     stp x4, x5, [sp, #16 * 2]
8     ...
9     stp x28, x29, [sp, #16 * 14]
10    str x30, [sp, #16 * 15]
11
12    // 将当前内核栈顶地址保存到原进程的 kern_ctx
13    mov x21, sp
14    str x21, [x0]
15
16    // 切换内核栈到目标进程的 kern_ctx
17    mov sp, x1
18
19    // 恢复通用寄存器 X0-X30
20    ldr x30, [sp, #16 * 15]
21    ldp x0, x1, [sp, #16 * 0]
22    ...
23    ldp x28, x29, [sp, #16 * 14]
24    add sp, sp, #ARCH_EXEC_CONT_SIZE
25

```

---



```
26 // 返回，此时返回目标进程调用该函数前所在地址
27 ret
28 END_FUNC(switch_to_process)
```

图 6.11 展示了 switch\_to\_process 过程中内核栈的变化过程。假设进程 p0 主动调用 process\_waitpid，然后调用 schedule 放弃 CPU 资源，此时调度器选择之前调用 process\_sleep 的进程 p1 作为下一个需要执行的进程（即切换的目标进程），那么切换将包含以下步骤。

1. 进程 p0 在 schedule（图中简写为 SC）中使用 bl 指令调用 switch\_to\_process，此时其返回地址寄存器 X30 会记录 switch\_to\_process 执行结束后应该返回的地址。

2. switch\_to\_process 执行与 exception\_enter 相似的逻辑，保存处理器上下文。由于该步骤只与内核中的状态相关，因此不需要保存前面提到的特殊寄存器和系统寄存器（SP\_EL0、ELR\_EL1、SPSR\_EL1）。

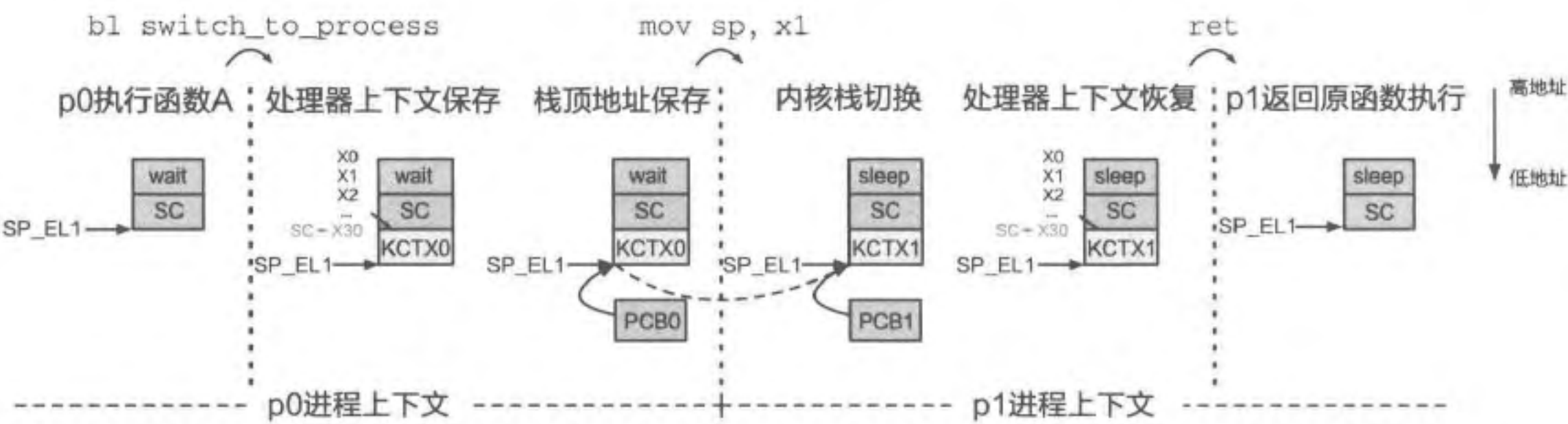


图 6.11 switch\_to\_process 前后栈结构的变化

3. switch\_to\_process 将当前的内核栈顶地址保存到原进程 PCB 的 kern\_ctx 中，然后将栈寄存器 SP\_EL1 切换到目标进程的 kern\_ctx，从而完成内核栈切换。

4. 目标进程 p1 执行与 exception\_exit 相似的逻辑，恢复通用寄存器，注意此时 p1 之前保存的 X30 也被恢复了。

5. 执行 ret 指令。由于 X30 已被恢复，因此此时会回到目标进程在调用 switch\_to\_process 之前保存的返回地址继续执行。由于 p1 之前调用了 schedule，那么此时也将回到该函数继续执行，从而完成了切换。

需要留意的是，switch\_to\_process 只完成了进程内核状态的切换，此时进程在用户态的处理器上下文仍然保存在其内核栈中，仍需要恢复。对于上述例子，当进程 p1 从 process\_sleep 中返回时，还需要调用 exception\_exit 恢复进程在用户态的处理器上下文，并执行 eret 返回用户态。而对于时钟中断的情况（图 6.8），也需要在调用 schedule 之后再调用 exception\_exit 完成处理器上下文恢复，才能正确

地返回用户态执行。

## 6.5 线程及其实现

### 本节主要知识点

- 什么是线程？为什么要有线程？
- 线程和进程的区别有哪些？
- 线程是如何实现的？
- 内核态线程和用户态线程都是什么？为什么要分成这两个类别？

### 6.5.1 为什么需要线程

通过实现进程及其相关接口，操作系统实现了对运行中程序的管理。但是，随着硬件技术的发展，“多核”架构逐渐成为主流，其中的每个 CPU 核心都能独立运行程序，但截至目前介绍的进程只能运行在单个核心之上，因而无法充分利用计算资源。假设小明参加了一场编程比赛，比赛要求编写一个 Web 服务器处理多个客户端发来的请求，在规定时间内处理最多请求的选手取得优胜。为了更快地处理用户请求，小明希望能充分利用自己电脑上的核心进行并行处理，但显然单一进程满足不了这个需求。

那么，小明能否采用创建多个进程的方式来并行处理请求呢？代码片段 6.30 给出了一种在 Linux 操作系统上可能的实现方案。每当获取一个新请求时，程序首先将网络包相关内容放入 packet 结构体中，然后调用 fork 创建新进程并处理网络包。由于新进程的虚拟内存空间中也包含 packet 中的数据，因此可以实现并行处理。

代码片段 6.30 基于 fork 的多进程并行处理程序示例（简化）

```
1 int main(int argc, char *argv)
2 {
3     while (TRUE) {
4         // 配置进程间通信
5         config_ipc(...);
6         // 获取网络包，放入 packet 结构体中
7         struct packet *pkt = recv_packet(...);
8
9         if (fork() == 0) {
10            // 子进程：实际处理网络包
11            struct result* res = handle_packet(pkt);
12            // 告知父进程处理已完成，然后返回
13            notify_complete(res);
14            return 0;
```